



Høyskolen
Kristiania

Session 2: Network and data communication

What is a network?

- Networks connect computers together
- This enables communication between people and systems over large distances.
- The predecessor to today's system of communication, called **the TCP/IP model**, was ArpaNET, which started with the interconnection of 4 universities in the United States - research funded through (D)ARPA.

Navn på laget	Betegnelse på overføringsenhet	Viktigste oppgaver/funksjoner Eksempel på protokoller/standards
Applikasjonslaget	Melding (Message)	Støtte nettverksapplikasjoner Ex: HTTP, DNS, FTP, SMTP, POP3....
Transportlaget	Segment	Transport av applikasjonslagsmeldinger mellom klient- og tjener-sidene til en applikasjon; herunder mux/demux, ulike nivåer av pålitelighet med mer Ex: TCP, UDP,...
Nettverkslaget	Datagram	Routing av datagram fra/til vertsmaskin gjennom nettverkskjernen Ex: IP (v4 og v6), ICMP, RIP, OSPF, BGP,...
Datalinjelaget	Ramme (Frame)	(Pålitelig) Levering av ramme fra nabo-node til nabo-node Ex: Ethernet II, FDDI, IEEE 802.11 ...
Fysisk	Bit	(Kode og) Flytte enkeltbit mellom kommunikasjonspartnere. Ex: 10BaseT, ...



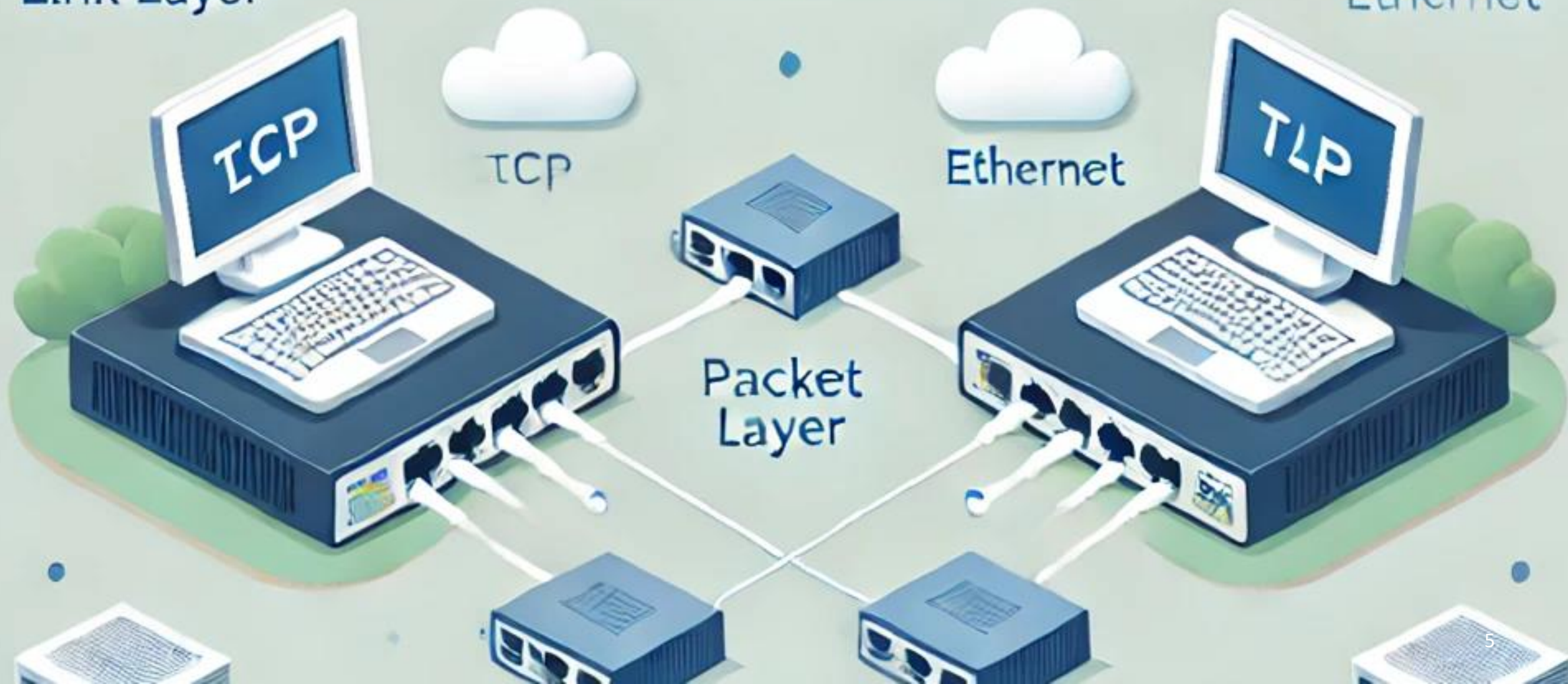
Link Layer

The Link Layer

in the TCP/IP model



Ethernet



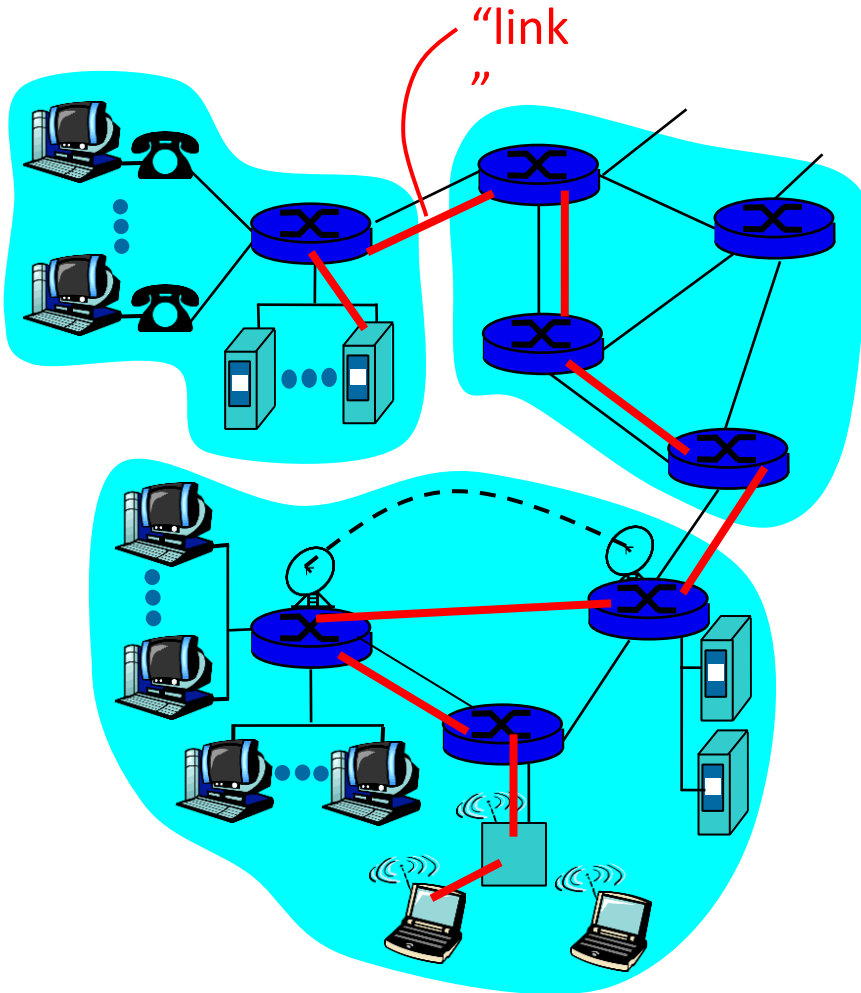
The Link Layer: Introduction

A little terminology:

- machines, routers and switches are nodes
- communication channels that connect neighboring nodes along the communication path are links
 - wired (wired) links
 - wireless (radio) links
- LAN = Local Area Network
- The data is called a frame, encapsulates datagrams

The link layer is responsible for transmitting datagrams from one node to another **neighbor** node over a link

Link between neighboring nodes



transportation analogy

- trip from Halden to Trondheim
 - train : Halden to Gardermoen
 - flight : Gardermoen to Værnes
 - bus : Værnes to Trondheim
- traveler = datagram
- transport stage = communication link
- transport type = link layer protocol
- travel agency = routing algorithm

Link layer services (1)

- **Framing and link access:**
 - encapsulation of datagrams into frames, adding header and trailer
 - channel access if shared medium (MAC = medium access control)
 - **MAC addresses** are used in frame headers to identify sender and receiver.
 - different from IP addresses!
- **Reliable delivery between neighboring nodes**
 - not necessary on links with low bit error rate (fiber and some types of copper cable)
 - wireless links: high bit error rate

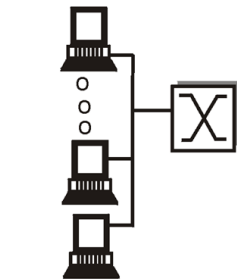
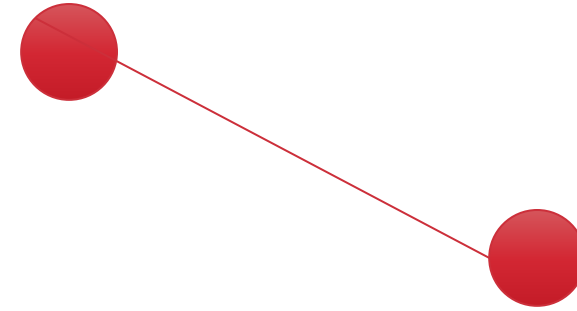
Link layer services (2)

- *Flow control:*
 - The sender must not send faster than the receiver can receive.
- *Error detection :*
 - errors are caused by attenuation and electrical noise
 - receiver detects bit errors:
 - either tells the sender to retransmit
 - or it just throws the frame
- *Error correction:*
 - receiver identifies *and corrects* bit errors without the sender having to retransmit
 - (Available in 802.11n)
- *Half-duplex and full-duplex*
 - with half-duplex, nodes at both ends of the link can transmit, but not both at the same time
 - with full-duplex both can send and receive at the same time

Links and protocols for multiple access

Two main types of “links”:

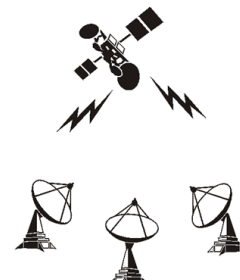
- **point-to-point**
 - PPP for dial-up access
 - point-to-point link between two routers
- **broadcasting** (shared medium)
 - traditional ethernet
 - 802.11 wireless LAN



shared wire
(e.g. Ethernet)



shared wireless
(e.g. Wavelan)



satellite



cocktail party

LAN addresses (= MAC addresses)

32-bit IP address:

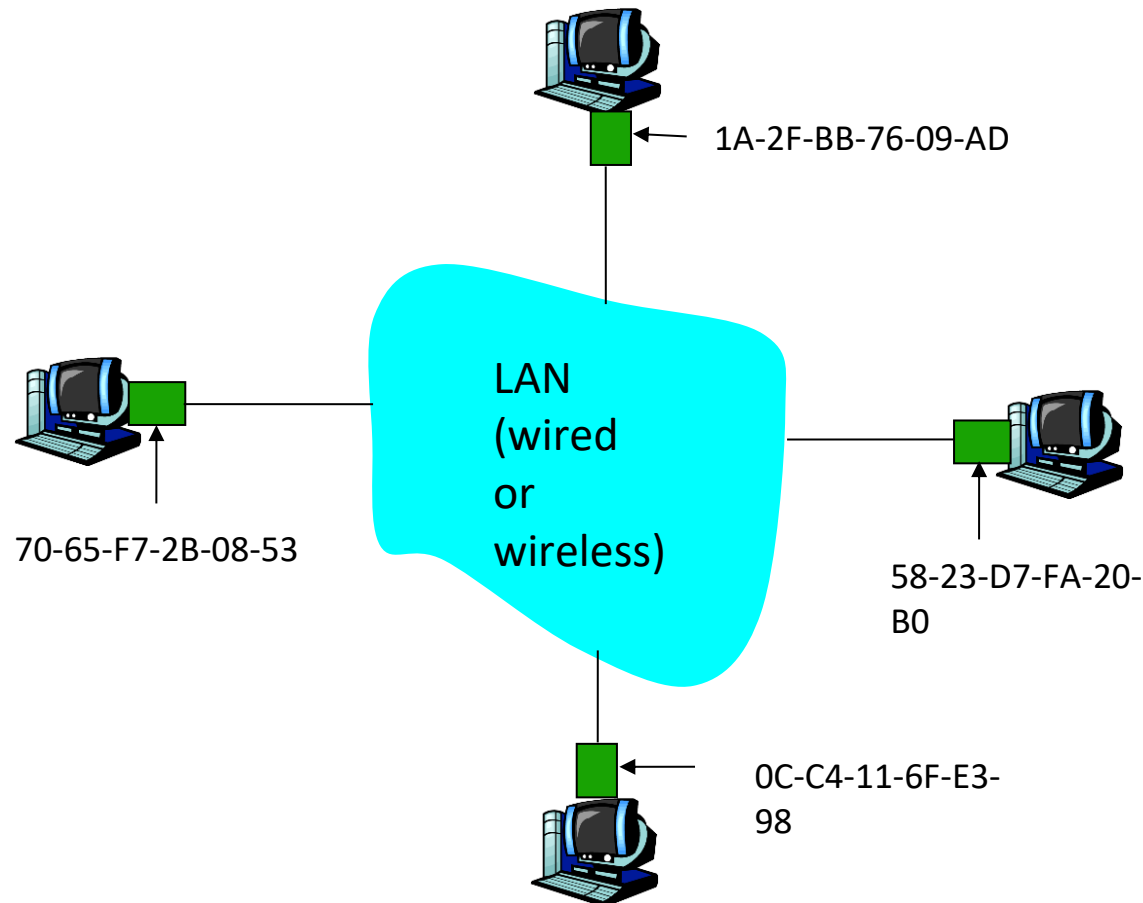
- *network layer* address
- used to get the datagram from your IP network to the recipient's IP network

MAC- (or “LAN-” or “physical-” or “ethernet-”) address:

- used to deliver a frame from one interface to another interface **on the same network**
- **48 bit** MAC address (for most LANs) burned into the network card's ROM

MAC addresses

Each network card has a “unique” MAC address



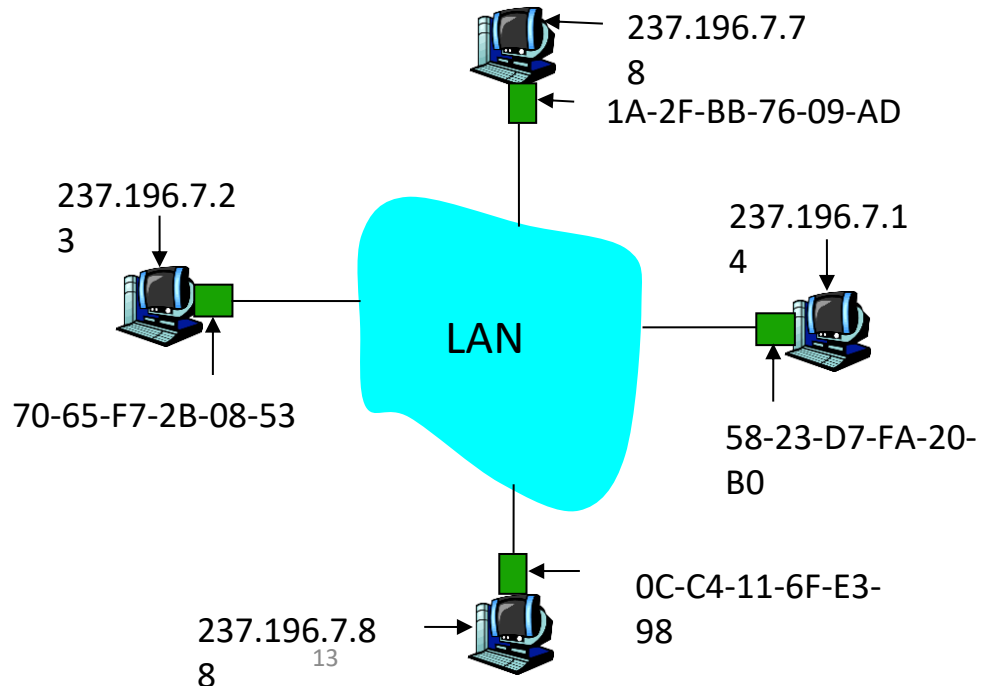
The broadcast address is
FF-FF-FF-FF-FF-FF
Multicast addresses have a trailing 1 in the first byte of the MAC address, and are usually treated as broadcast, e.g.
A3-05-77-31-27-BA

 = network card

ARP : Address Resolution Protocol

How to find the MAC address of a node of which

you know the IP address?



- Every IP node (computer and router) on a LAN has an **ARP** table/cache

- ARP table: IP/MAC address mappings for some LAN nodes

<IP address; MAC address; TTL>

- TTL (Time To Live): the time the mapping should remain in the ARP table (typically 20 min)

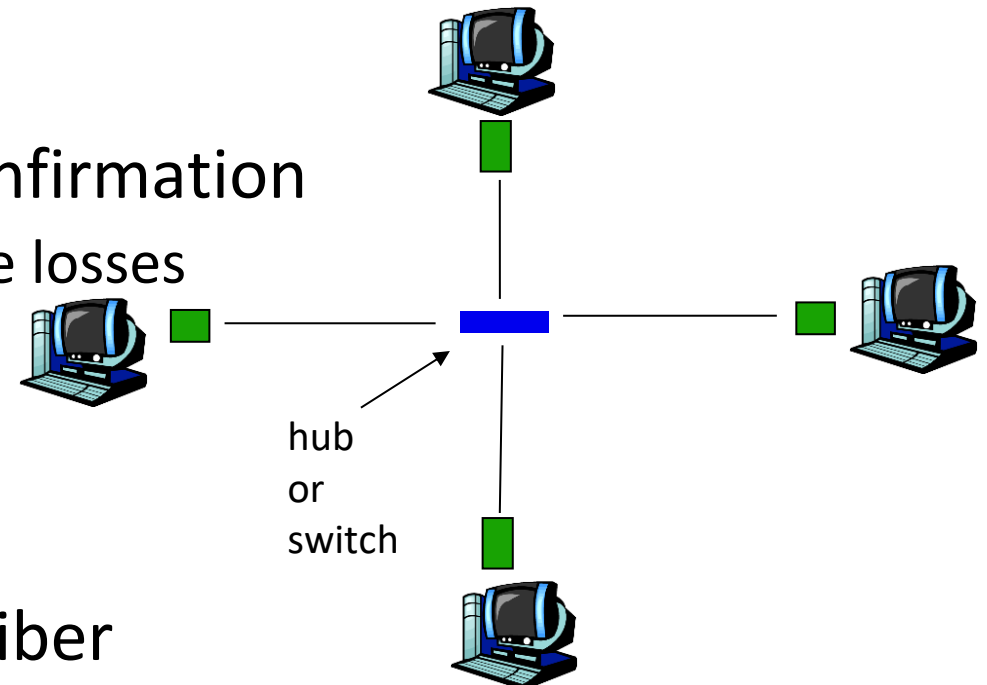
ARP : Address Resolution Protocol

- A wants to send a datagram to B and knows B's IP address
 - Suppose B's MAC address is not in A's ARP table
- A **broadcasts** an ARP request containing B's IP address
 - all machines on the LAN receive the ARP request
- B also receives the ARP packet and replies to A with its MAC address
 - frame is sent directly to A's MAC address
- A caches (stores) the IP-to-MAC address pair in its ARP table until the information becomes stale
 - “soft state”: information that disappears if it is not refreshed
- ARP is “plug and play”:
 - a node creates its ARP table without any help from anyone

Star topology

- There are several ways to connect machines, for example in a series (bus topology)
- Today, star topology is used almost exclusively

- **Unreliable:** recipient does not send confirmation
 - datagrams to the network layer may have losses
 - transport layer can repair (TCP)
 - the application layer can repair



- At the physical layer: 100BaseT, GbE, fiber

Switch



- Link layer box
 - buffers and forwards Ethernet frames
 - checks frame header and forwards **when needed** – based on recipient's MAC address
 - when the frame is to be forwarded on a segment, CSMA/CD is used
- transparent
 - machines are unaware of the switch's presence
- plug-and-play, self-learning
 - switches do not need to be configured



THE NETWORK LAYER

NETWORK LAYER

TECR — LAYER

NETWORK

continued

CHAPTER

...and

4E31E6C8-0000

VP

REFERENCES



NETWORK LAYER

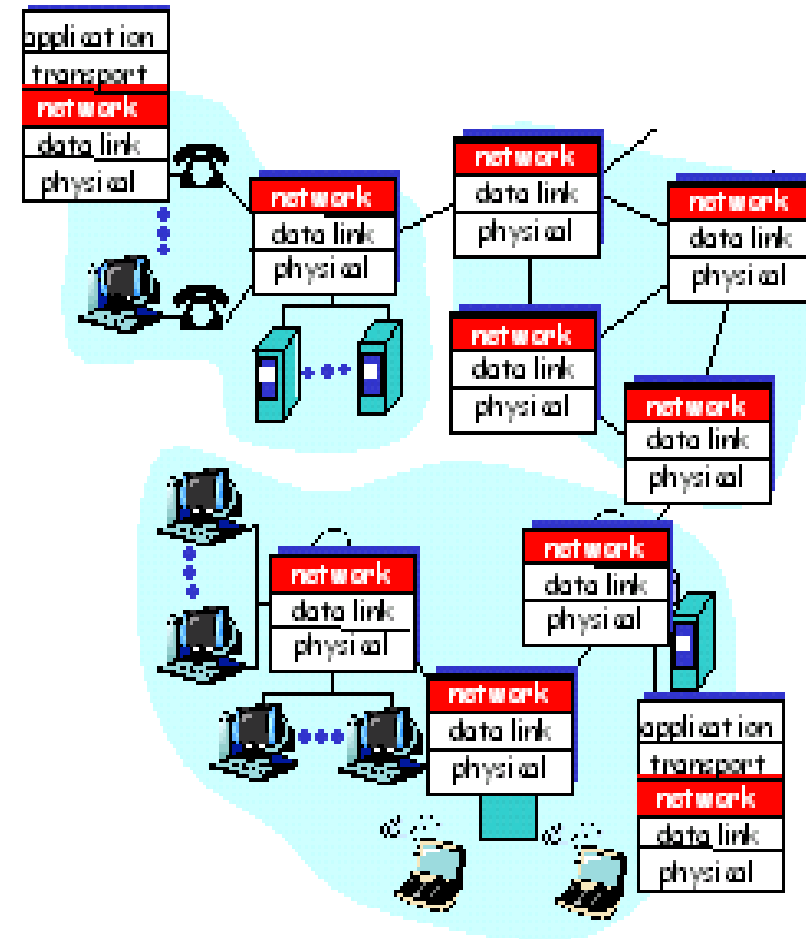


NETWORK LAYER



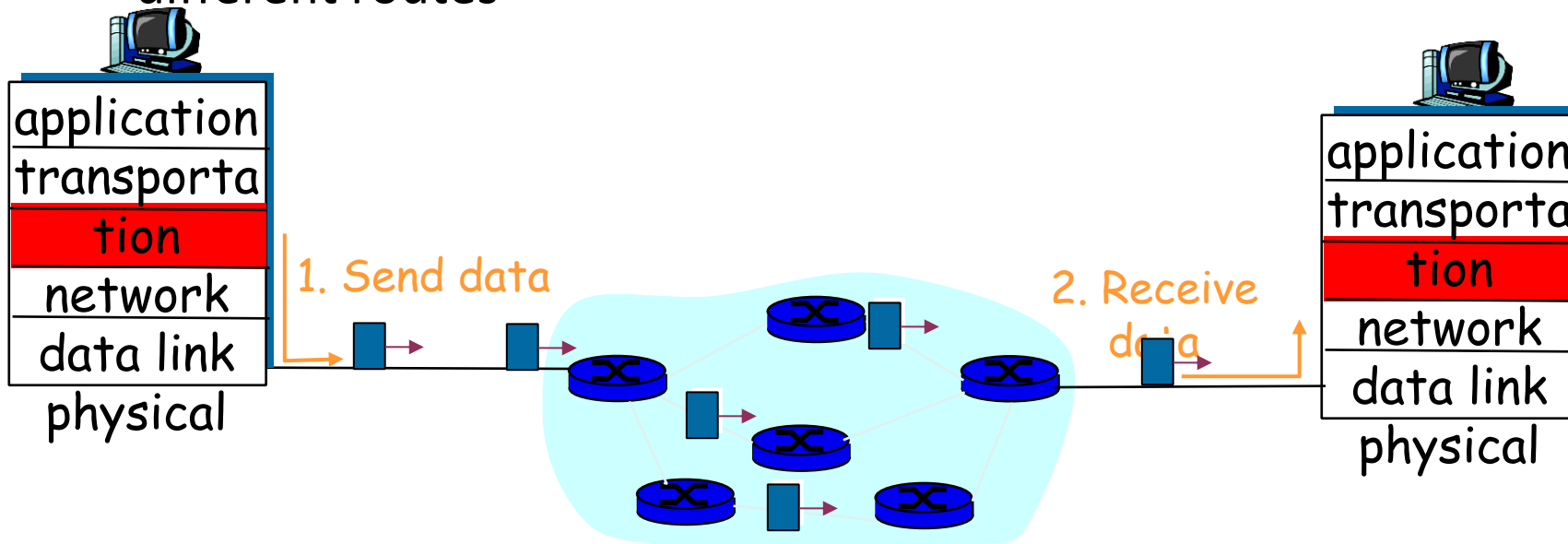
The Network Team

- Moves packets from sender to receiver
- Network protocol also on every stopover
- Routing from sender to receiver
- Switching packets from the router's input side to the router's output side
- If necessary, router call setup is defined for the entire route before the packet is sent

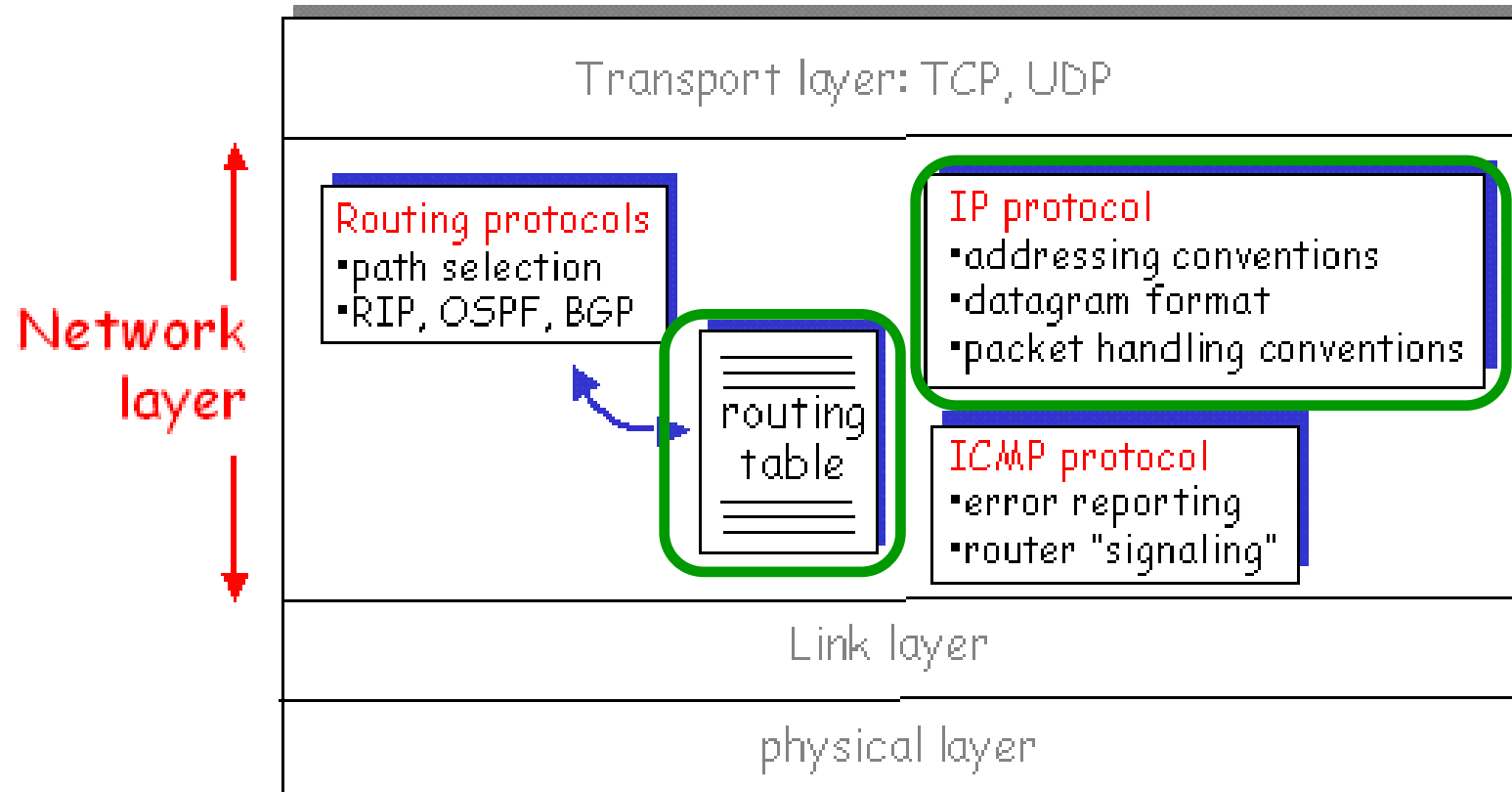


Datagram network (Internet)

- No pre-configuration at the network layer level
- The routers do not care about the state *of the route*
 - Stateless routers
- The packets are routed based on the recipient ID.
 - Packets between the same sender and receiver **can** follow different routes

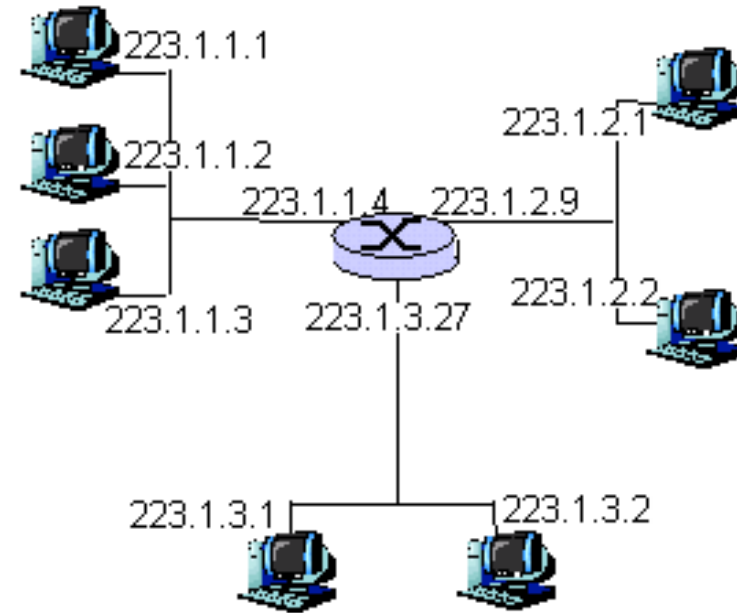


Internet network layer



IPv4 addressing

- IPv4 address: 32-bit "id" for each host machine and router interface (adapter)
- A host machine can have several interfaces
- A router usually has multiple connections, each with its own interface.
- IP address belongs to each interface



223.1.1.1 = $\underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$

ipconfig

ipconfig shows the network parameters for the interfaces/adapters

```
C:\Users\blislog>ipconfig
Windows IP Configuration

Ethernet adapter e0:

    Connection-specific DNS Suffix  . : 
    IPv6 Address . . . . . : 2001:700:2e00::51
    Link-local IPv6 Address . . . . . : fe80::b46c:b98f:85ec:dba0%12
    IPv4 Address. . . . . : 158.36.131.51
    Subnet Mask . . . . . : 255.255.255.128
    Default Gateway . . . . . : 2001:700:2e00::1
    158.36.131.1
```

IPv4 address
& Netmask
= Network prefix
from which it is routed;
Std Gateway = the way out
into the Internet

```
Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
```

```
Tunnel adapter isatap.{84470FB0-16A7-4A31-B6B9-8AECF834115C}:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
```

```
Tunnel adapter Teredo Tunneling Pseudo-Interface:
```

```
    Connection-specific DNS Suffix  . : 
    IPv6 Address . . . . . : 2001:0:5ef5:73b8:3826:3138:61db:7ccc
    Link-local IPv6 Address . . . . . : fe80::3826:3138:61db:7ccc%15
    Default Gateway . . . . . :
```

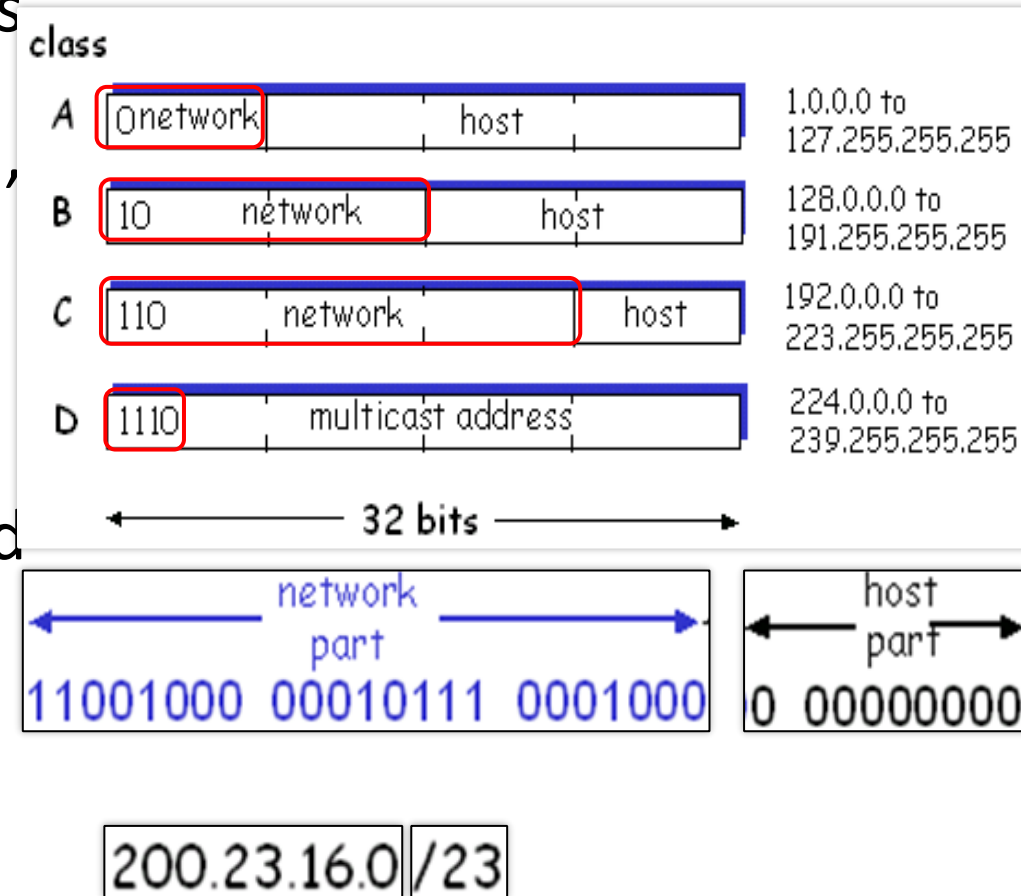
```
Tunnel adapter isatap.{C682A4AE-3FEC-4053-8456-5A377FD0FF55}:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
```

Techniques for
sending IPv6
through IPv4
networks

IP addresses: classes and CIDR

- Originally divided into 6 different classes each with a predefined **prefix** length
- Address classification became too “rigid”
 - A class may risk containing (many) unused addresses
- Class A, B, C are regular addresses, D is multicast, E is reserved for research, and 127.* is a reserved "class" for loopback.
- Classless Inter-Domain Routing (CIDR)
 - The network part has arbitrary length, x
 - Format abcd/x



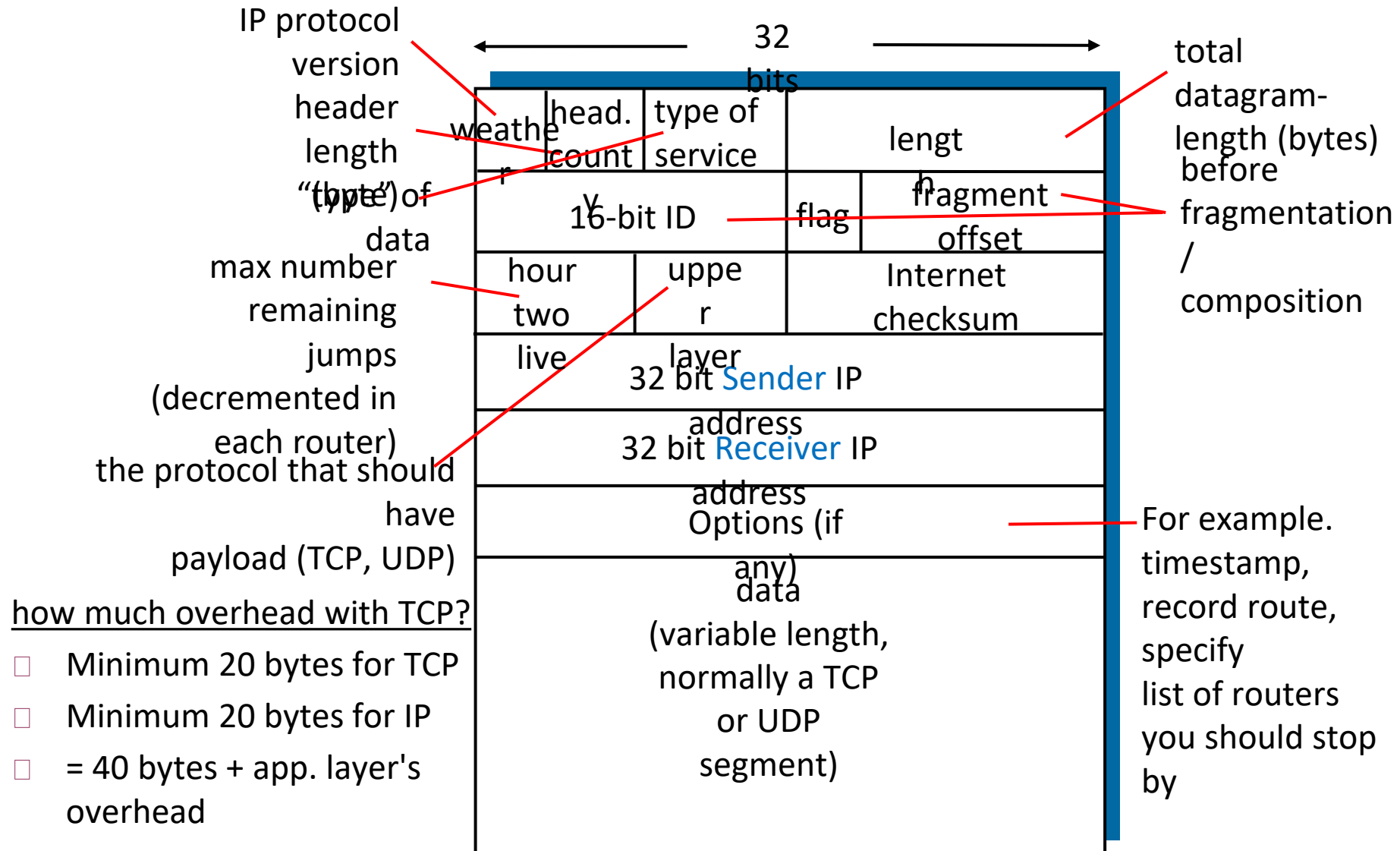
IPv4 addresses: assignment

- For hosts in LAN
 - Can be set manually/statically
 - **Dynamic Host Configuration Protocol** (DHCP)
- For networks
 - Gets assigned their share of the ISP's allocated address space
- For Internet Service Providers (ISPs)
 - International body (**ICANN**) assigns addresses, manages DNS, assigns domain names, and resolves disputes
 - “The Continental Registrar”: **RIPE** distributes IP addresses and AS numbers to Europe and others.

Dynamic Host Configuration Protocol

- Each DHCP server has a set of possible addresses (pool)
- Sets the address dynamically with "plug-and-play"
- Host sends : DHCP discover
- DHCP server responds : DHCP victim
- Host machine sends : DHCP request
- DHCP server sends : IP address and other network parameters (e.g. DNS server) + DHCP ack
- Host machine is set up with these values

IPv4 datagram format



IP Address & Netmask = IP Network

- Machines/adapters must belong to the same IP network to be able to send directly to each other
 - 10.21. 3.5 / 255.255. 254 .0 can send directly to 10.21. 2.255 / 255.255. 254 .0
 - 10.21. 3 .5 / 255.255. 255 .0 must send via **gateway** (router) to reach 10.21. 2 .255 / 255.255. 255 .0
- **The prefix** is determined by the IP address and the netmask, and it is this that determines whether you belong to the same IP network or not.

Math example: Which network?

10.21.26.184 with netmask
255.255.252.0 belongs to which network?

```
10 . 21.0001 10 10.1011 1000
& 255.255.1111 11 00.0000 0000
-----
10 . 21.0001 10 00.0000 0000
22 bits for prefix , 10 bits for host
```

The network is **10.21.24.0/22**

Lowest address is **10.21.24.1**

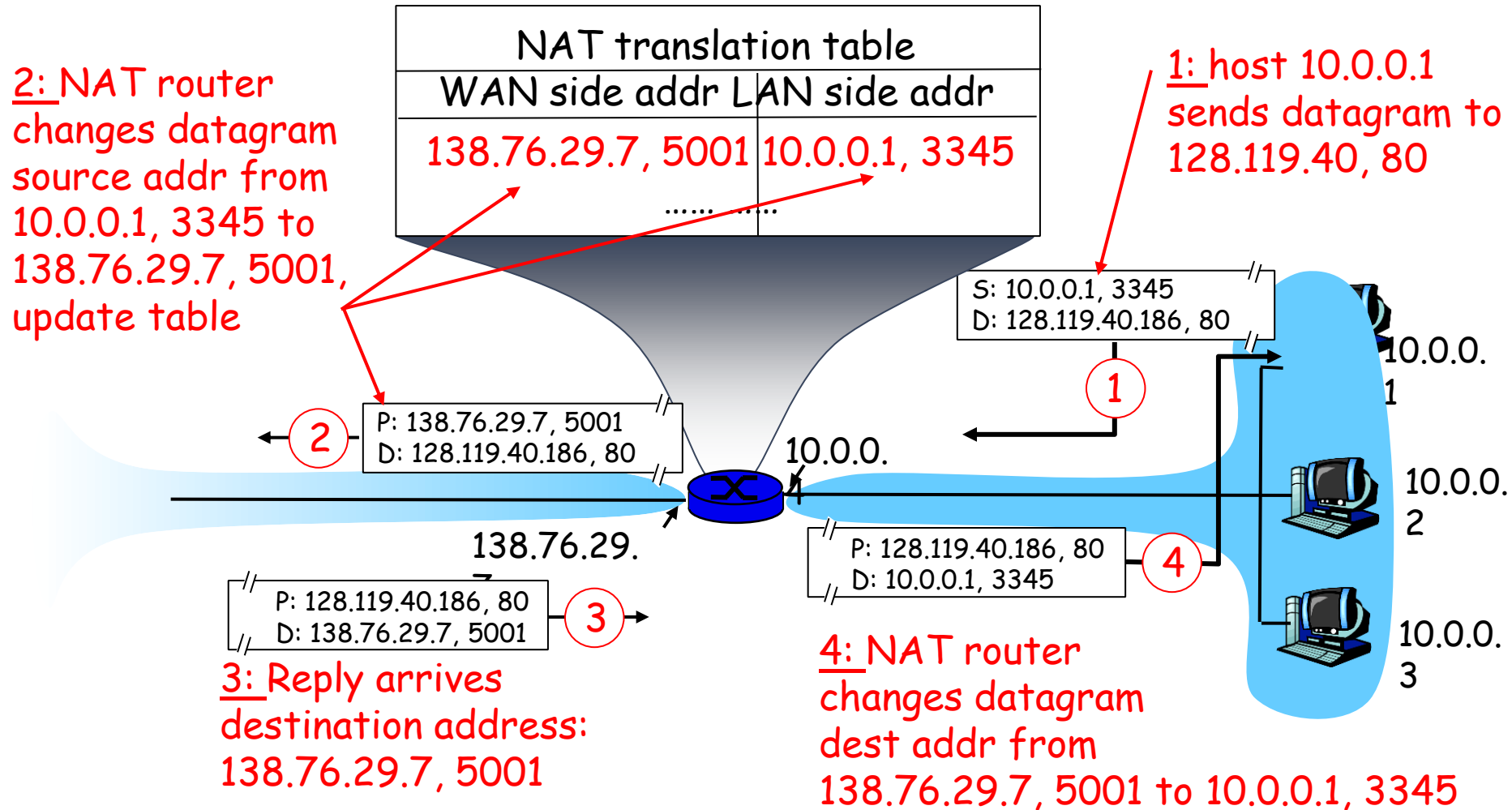
Broadcast is **10.21. 27.255**

all host bits set to 1!!!

NAT: Network Address Translation

- **Why?:** The LAN only has one/a few IP addresses from the Internet's perspective:
- ISP does not have to allocate an address range:
 - only one/a few IP address(es) for an entire organization's network
- Can change addresses within the LAN without having to inform the outside world about it
- Can change ISP without having to change addresses in the LAN
- Devices in the LAN are directly addressable or visible to outsiders (better security)

NAT: Network Address Translation



UPDATE

D Some special IP addresses

Some IP addresses are reserved for special uses:

- *Private addresses* are only used within a WAN
 - cannot be routed outside LAN/WAN
 - automatically dropped by Internet routers

IPv4 addresses	Network
10 .0.0.0 – 10.255.255.255	1 class A network
172.16 .0.0 – 172.31 .255.255	16 class B networks
192.168 .0.0 – 192.168.255.255	65536 class C network

- **169.254 .1.0 – 169.254 .254.254** Broadcast (also not routable)
- **0.0.0.0** Used as a temporary address after startup
- **127.0.0.1** (loopback address, traffic is only sent back to your own machine)

Link
layer



Transport



Network
Layer



TRANSPORT
LAYER



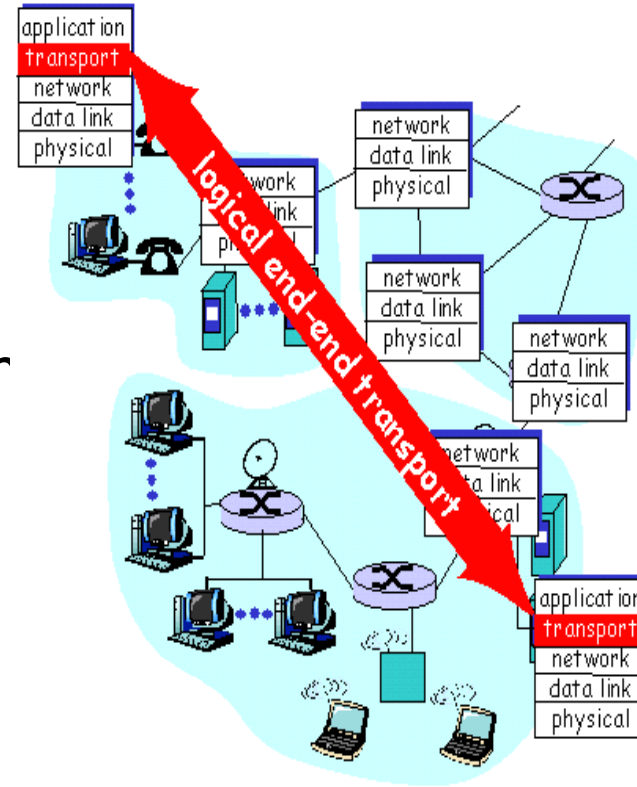
TCP

TCP

UDP

Transportation service

- Sets up *logical* communication between application processes on different clients
- The transport protocol is run on each *end* system
- Transport layer protocol
 - Data transfer between *processes*
- Uses Internet Protocol (IP)
 - Data transfer between *systems* (host machines)



Transport-layer protocols

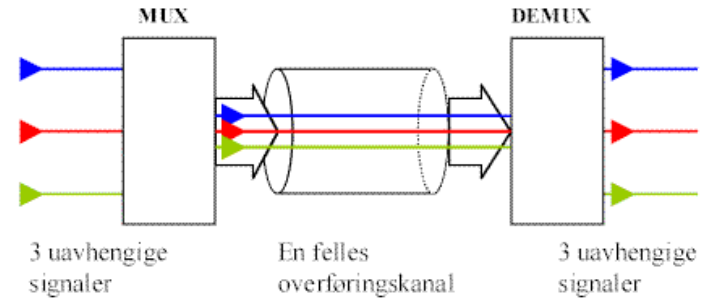
The Internet uses [the network](#) protocol [IP](#); it does its best, but makes no guarantees - "best effort"

- UDP
 - Sends a [datagram](#) , which may consist of several parts, to the recipient and hopes it arrives
 - Improves [IP](#) only with [end-to-end](#) control and [error-checking](#)
- TCP
 - Establishes a "permanent" connection
 - Adds [flow control](#) ,
[sequence](#) number,
[acknowledgement](#) ,
[time control](#) ,
[error checking](#) and
[traffic jam](#) control (saturation control)

Multiplexing / demultiplexing

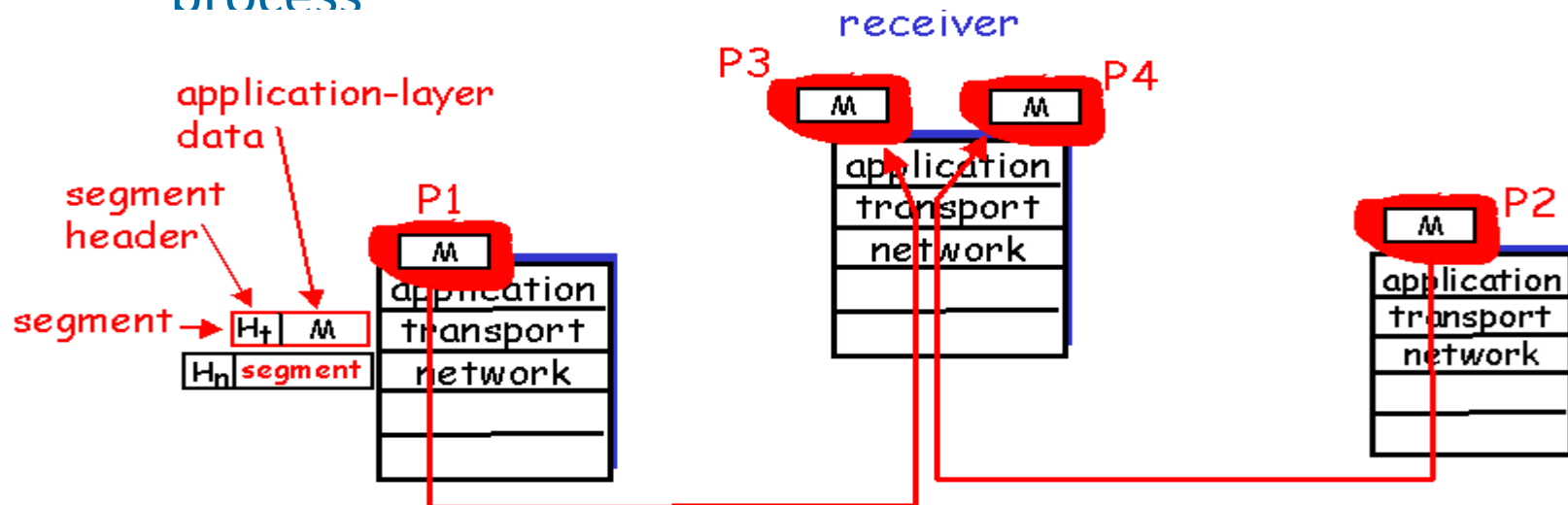
- Segment

- Data unit exchanged between transport layers
- TPDU (Transport Protocol Data Unit)



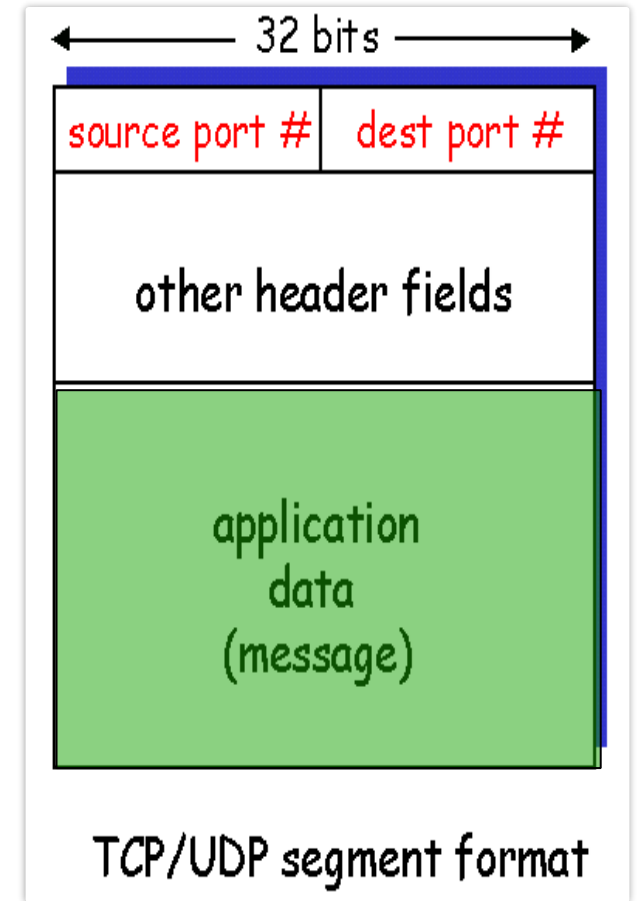
- Demultiplexing

- Deliver received segments to the correct process



Multiplexing <- port number

- Collects data from application processes and packages it with a header
- The head contains **the transmitter** and **receiver port number**
- Port number = **16 bit** unsigned integer
- Ports **0-1023** are "**well known**" (RFC 1700)
 - Secure Shell: port **22**
 - SMTP: port **25**
 - DNS: port **53**
 - HTTP: port **80**
 - HTTP over TLS/SSL: port **443**
- Other ports are divided into:
 - **Registered**
 - 1024-49151 (0x0400-0xBFFF)
 - Can be used for other purposes as well, but is **registered** for a service with IANA
 - **Private/Dynamic** :
 - 49152-65535 (0xC000-0xFFFF)



Transport layer

- The first main task solved at the transport layer is thus to multiplex/demultiplex from/to local processes and the common channel (Internet)!
- Port number acts as **ID number** for local and contacted process!

netstat

Tools where you can see all open ports (in practice):

```
C:\>netstat -n
Active Connections

Proto Local Address           Foreign Address         State
TCP   127.0.0.1:19872          127.0.0.1:55169        ESTABLISHED
TCP   127.0.0.1:27015         127.0.0.1:55155        ESTABLISHED
TCP   127.0.0.1:52965         127.0.0.1:52967        ESTABLISHED
TCP   127.0.0.1:52967         127.0.0.1:52965        ESTABLISHED
TCP   127.0.0.1:55155         127.0.0.1:27015        ESTABLISHED
TCP   127.0.0.1:55169         127.0.0.1:19872        ESTABLISHED
TCP   127.0.0.1:55169         127.0.0.1:19872        CLOSE_WAIT
TCP   Client IP address:      Server IP address:      Connection
TCP   port number             port number             Status
TCP   158.36.131.51:55812     174.36.30.34:80        ESTABLISHED
TCP   158.36.131.51:56109     158.36.131.26:443      ESTABLISHED
TCP   158.36.131.51:56892     74.125.71.125:443      ESTABLISHED
TCP   158.36.131.51:59550     212.161.8.5:12350      ESTABLISHED
TCP   158.36.131.51:60407     158.194.55.31:53718    ESTABLISHED
TCP   158.36.131.51:61306     174.129.195.86:443     CLOSE_WAIT
TCP   158.36.131.51:62305     74.125.79.118:443      ESTABLISHED
TCP   158.36.131.51:62434     158.36.191.141:443     ESTABLISHED
```

netstat is the command that provides an overview of open ports:

netstat -a : also UDP

netstat -s : statistics

netstat -r : routing table

netstat -n : not DNS name

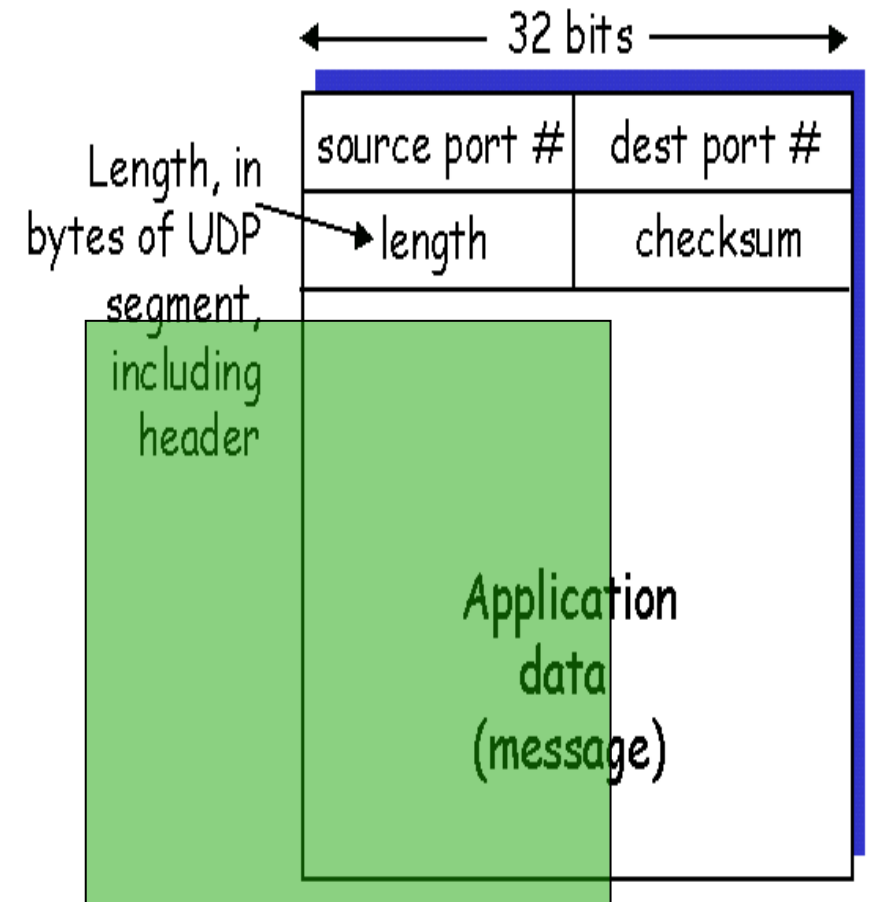
+ many more to see which process etc.
(-b on Win7 etc.)

UDP (User Datagram Protocol)

- Very simple Internet transport layer protocol
 - Segments may be **lost**
 - Segments may be delivered in **the wrong** order
 - **No handshake** between sender and receiver
- Why UDP?
 - **No connection** establishment - no delay
 - **Does not** set up any **common state** for sender and receiver
 - Small segment head
 - No traffic jam control
 - Enables **broadcasting**
 - If you need reliability, you can add it to the program at the application layer level.

UDP (User Datagram Protocol)

- Often used in connection with **multimedia** where the human brain can correct the errors
- Other uses
 - DNS
 - SNMP
- The recipient's **application** can handle error handling.

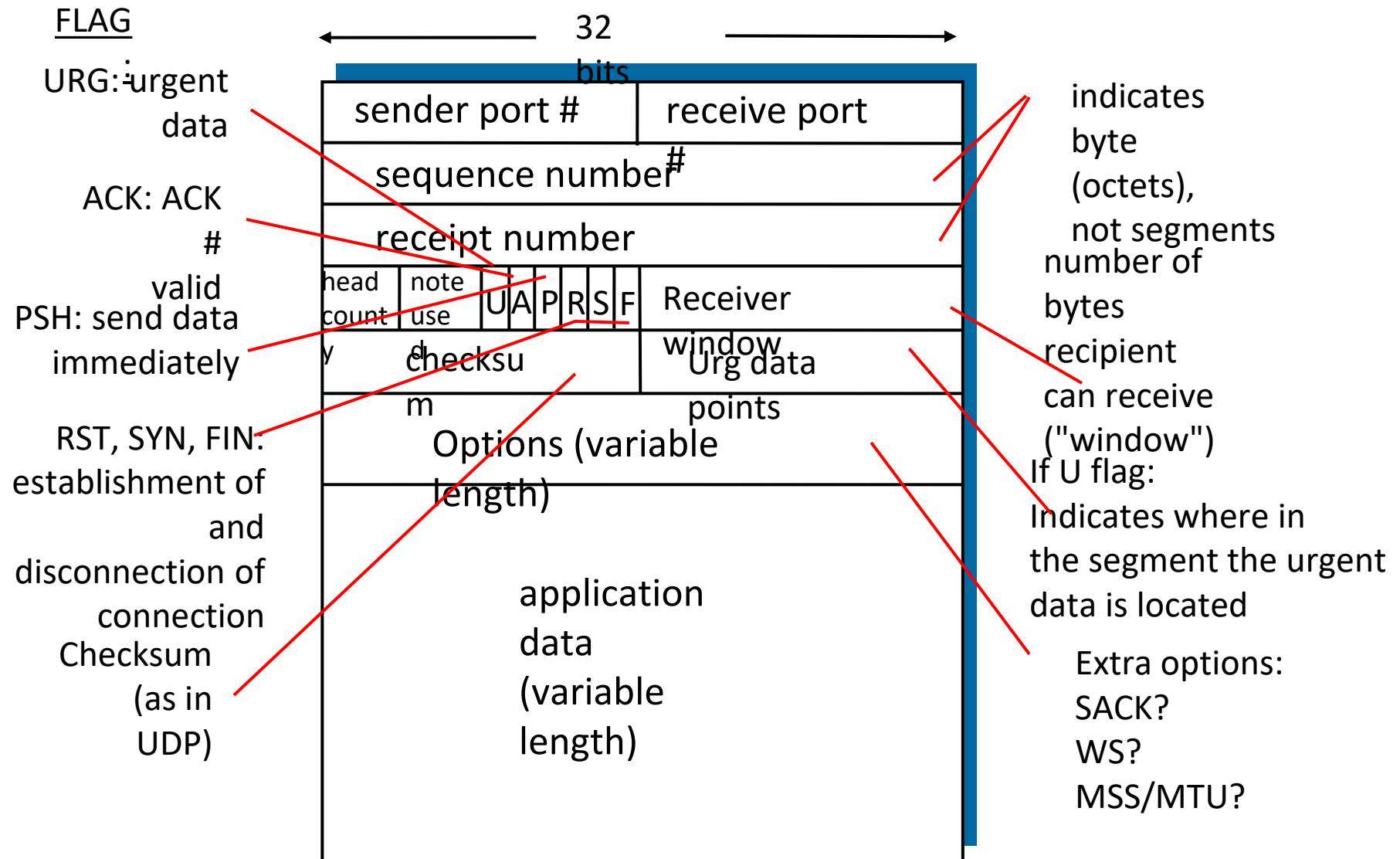


UDP segment format

TCP (Transmission Control Protocol)

- Point to point
 - One sender, one receiver
- Reliable , ordered byte stream
- Pipeline
 - Flow and saturation controls determine window size
- Sender and receiver buffer
- Full duplex data
 - Both can send and receive at the same time
- Connection-oriented
 - Handshake before data transfer
- Flow control
 - Sender does not drown receiver

Structure of TCP header



TCP: **Starting** the connection

- Sender and receiver establish a connection before data segments are exchanged
 - Initializing TCP variables
 - Sequence numbers, buffers, windows.....
- Client -> sender -> receiver -> server
 - Setting up socket
- Client sends a special TCP segment with **SYN**
 - SYN flag in header set
 - Specifies the starting sequence number
- Server responds with **SYN ACK**
 - SYN and ACK flags in the header set
 - Sets up start sequence number, buffers, windows etc.

Wireshark (tool for TCP and IP)



1)
Experiment

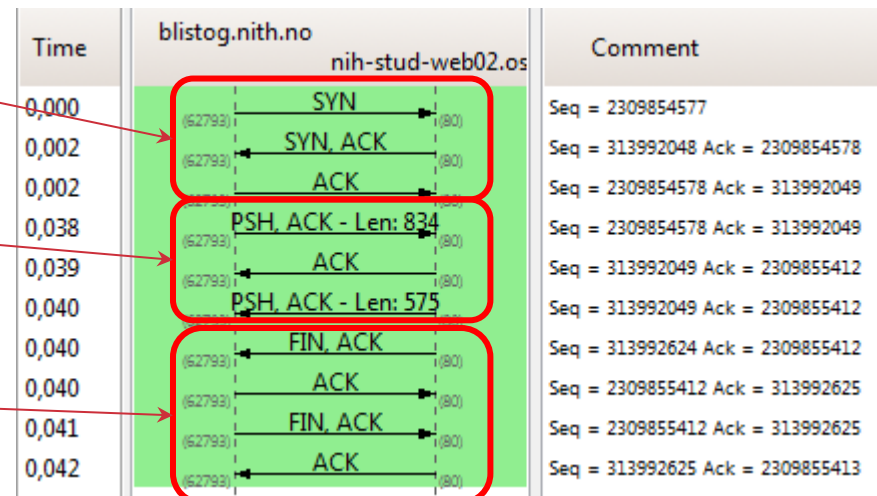
2)
Result

Time	Source	Destination	Protocol	Info
5 3.012370	blistog.nith.no	nih-stud-web02.osl	TCP	62793 > http [SYN] Seq=2309854577 win=8192 Len=0 MSS=1460 WS=2 SACK_PERM=1
6 3.014275	nih-stud-web02.osl	blistog.nith.no	TCP	http > 62793 [SYN, ACK] Seq=313992048 Ack=2309854578 win=5840 Len=0 MSS=1380
7 3.014329	blistog.nith.no	nih-stud-web02.osl	TCP	62793 > http [ACK] Seq=2309854578 Ack=313992049 win=16560 Len=0
8 3.050158	blistog.nith.no	nih-stud-web02.osl	HTTP	GET /~blistog/lab0.html HTTP/1.1
9 3.051843	nih-stud-web02.osl	blistog.nith.no	TCP	http > 62793 [ACK] Seq=313992049 Ack=2309855412 win=59 Len=0
10 3.052514	nih-stud-web02.osl	blistog.nith.no	HTTP	HTTP/1.1 200 OK (text/html)
11 3.052515	nih-stud-web02.osl	blistog.nith.no	TCP	http > 62793 [FIN, ACK] Seq=313992624 Ack=2309855412 win=59 Len=0
12 3.052571	blistog.nith.no	nih-stud-web02.osl	TCP	62793 > http [ACK] Seq=2309855412 Ack=313992625 win=16416 Len=0
13 3.052912	blistog.nith.no	nih-stud-web02.osl	TCP	62793 > http [FIN, ACK] Seq=2309855412 Ack=313992625 win=16416 Len=0
14 3.054161	nih-stud-web02.osl	blistog.nith.no	TCP	http > 62793 [ACK] Seq=313992625 Ack=2309855413 win=59 Len=0

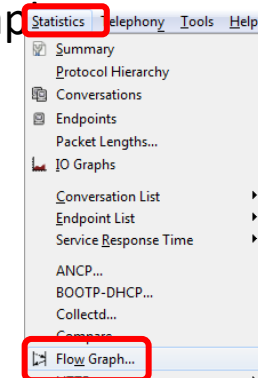
Connectio
n

Data transfer
GET + ACK +
200OK

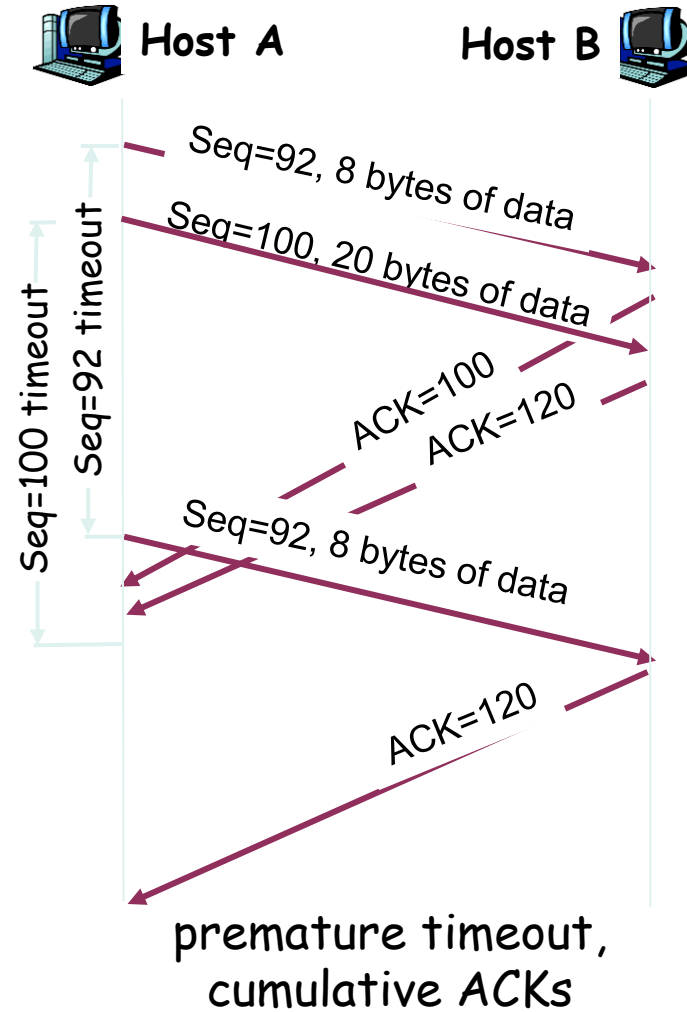
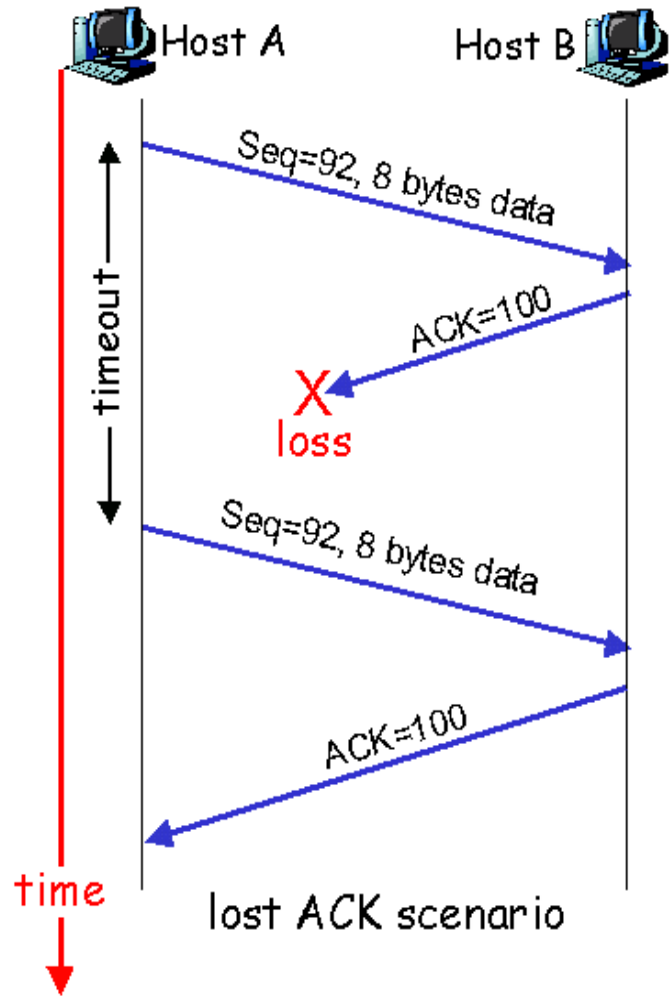
Disconnectio
n



3) Flow
Graph



Resending



I wanted to tell you all
a joke about UDP.

*But I don't know if
you would get it...*



Høyskolen
Kristiania